

BANK CUSTOMER CHURN PREDICTION PIPELINE

End-to-End Machine Learning Project Report

Project Type	Machine Learning / Predictive Analytics
Model	XGBoost Classifier
Explainability	SHAP
Backend	FastAPI
Frontend	Streamlit
Data Size	10,000 customer records

A complete pipeline covering data ingestion, exploratory analysis, preprocessing, class balancing, model training, interpretability, serialization, API serving, and an interactive user interface.

1. Executive Summary

This project presents an end-to-end machine learning pipeline designed to predict bank customer churn. The system progresses from raw data ingestion and exploratory data analysis through feature engineering, class balancing, model training, evaluation, interpretability, and production-style deployment.

An XGBoost classifier was trained using SMOTE-balanced training data to address the significant class imbalance between retained and churned customers. SHAP (Shapley Additive exPlanations) was incorporated to explain the model's predictions and identify important business drivers of churn.

For deployment, the solution uses a decoupled full-stack architecture. A FastAPI backend exposes a REST API for prediction, while a Streamlit frontend provides a form-based interface for bank users such as managers. The resulting system can accept customer information and return a churn probability together with a categorical risk level.

2. Development Environment & Architecture Setup

The project was developed in an isolated Python virtual environment using a modular directory structure. The structure separates raw data, source code, notebooks, and serialized machine learning artifacts, making the project easier to maintain and deploy.

Core Technology Stack

Area	Technologies
Data Processing	pandas, numpy, scikit-learn, imbalanced-learn
Modeling	XGBoost 1.7.6
Explainability	SHAP
Backend	FastAPI, Uvicorn, Pydantic
Frontend	Streamlit
HTTP Communication	requests
Development	PowerShell, Jupyter Notebook, Python scripts

High-Level Architecture

Raw Customer Data → EDA → Preprocessing → SMOTE → XGBoost → SHAP → Model/Scaler Serialization → FastAPI REST API → Streamlit Dashboard

The architecture separates the prediction service from the user interface. This allows the model-serving layer to operate independently and enables other clients to consume the same API in the future.

3. Data Ingestion & Exploratory Data Analysis

The dataset contains 10,000 customer records and 12 initial columns. Initial data-quality checks confirmed that the dataset contained no missing values, allowing preprocessing to proceed without imputation.

Key EDA Findings

- Class imbalance: 79.63% of customers were retained while 20.37% were classified as churned.
- Demographic trend: Customers in the 40–50 age bracket showed a noticeably higher churn rate than younger demographics.
- Geographic trend: Customers in Germany displayed a disproportionately high churn rate compared with customers in France and Spain.

EDA Metric	Observation
Total Records	10,000
Initial Columns	12

Missing Values	None detected
Retained Customers	79.63%
Churned Customers	20.37%

4. Data Preprocessing & Feature Engineering

1. Feature Selection: The customer_ identifier column was removed because it does not provide useful predictive information.

2. Categorical Encoding: One-Hot Encoding was applied to country and gender with drop_first=True, producing country_Germany, country_Spain, and gender_Male.

3. Data Splitting: An 80/20 train-test split was performed using stratification on the target variable to preserve the class distribution.

4. Feature Scaling: StandardScaler was fitted only on the training data and then applied to the test data, preventing data leakage.

5. Class Balancing: SMOTE was applied exclusively to the training set, producing a balanced training dataset with 6,370 samples in each class.

Data Leakage Prevention

A key design consideration was ensuring that information from the test set did not influence model training. The scaler was fitted only on training data, and SMOTE was performed only after the train-test split. Therefore, synthetic minority samples were generated without contaminating the unseen test set.

5. Model Training & Evaluation

An XGBoost Classifier was trained on the SMOTE-balanced training dataset using eval_metric='logloss'. The final evaluation was performed on the original, unseen test set rather than on the resampled data.

Evaluation Metric	Result
ROC-AUC	0.8380
Overall Accuracy	85%
Churn Customers Identified	229

The evaluation demonstrates that the model provides useful discrimination between churn and non-churn cases. The identified churn cases can be used as a starting point for proactive customer-retention workflows.

6. Model Interpretability — SHAP Analysis

SHAP (Shapley Additive exPlanations) was used to make the XGBoost model more interpretable. Instead of treating the model as a black box, SHAP values provide information about how individual features contribute to predictions.

Feature	Observed Relationship / Interpretation
Age	Higher age values are strongly associated with increased churn probability.
Number of Products	The relationship is non-linear; very low or very high product counts can act as strong churn signals.
Active Membership	Active membership (value = 1) acts as a major protective factor and pushes predictions toward retention.

7. Model Serialization & Deployment

Model Artifacts

The trained XGBoost model and fitted StandardScaler were serialized using joblib and stored in the /models directory. These artifacts allow the production application to load the trained components without retraining the model each time the server starts.

FastAPI Backend

The FastAPI backend exposes a RESTful /predict endpoint. Pydantic models provide strict validation of incoming JSON requests. The backend applies the same preprocessing logic used during training, passes the processed input to the serialized XGBoost model, and returns a churn probability and categorical risk level.

Streamlit Frontend

The Streamlit application provides a responsive form-based dashboard for end users such as bank managers. Users can enter customer profile information, submit it to the FastAPI backend through an HTTP POST request, and view the resulting churn risk through progress indicators and alert-style messages.

8. End-to-End Prediction Workflow

- 1. User enters customer information in the Streamlit interface.
- 2. Streamlit constructs a validated JSON payload.
- 3. The payload is sent to the FastAPI /predict endpoint.
- 4. FastAPI validates the request using Pydantic.
- 5. Production preprocessing transforms the input into the model's expected feature representation.
- 6. The serialized XGBoost model generates a churn probability.
- 7. The API converts the probability into a categorical risk level.
- 8. Streamlit displays the prediction using interactive UI elements.

9. Business Value

The system is designed to support proactive customer-retention activities. Instead of relying solely on historical reporting, bank teams can use predicted churn probabilities to identify customers who may require attention. SHAP explanations additionally provide feature-level context that can help users understand why a customer received a particular prediction.

The decoupled API architecture also makes the solution extensible: the same prediction service can potentially be consumed by a web application, mobile application, batch-processing system, or another enterprise service.

10. Conclusion

This project demonstrates a complete machine learning lifecycle for bank customer churn prediction. It combines careful preprocessing and leakage prevention, SMOTE-based class balancing, XGBoost classification, SHAP-based interpretability, model serialization, REST API deployment, and an interactive Streamlit frontend.

The resulting pipeline moves beyond model development into a practical prediction system that can accept new customer information and provide both a churn-risk estimate and interpretable signals for business users.